PROPERTY MANAGEMENT SYSTEM

Technical Architecture & Implementation Documentation

Technology Stack

Java 17 • Spring Boot 3.x • MySQL 8.0 • JPA/Hibernate
Liquibase • ActiveMQ • JWT RS256 • MapStruct

Version 1.0 | Production-Ready Architecture

1. System Overview

1.1 System Purpose

The Property Management System (PMS) is an enterprise-grade, multi-tier web application designed to digitize and streamline end-to-end residential and commercial property management operations. It enables property management companies to manage their entire portfolio — from onboarding tenants and executing leases, to collecting rent, handling maintenance, and generating financial reports — through a unified, role-aware platform.

1.2 Target Users

Role	Description	Key Responsibilities
System Admin	Top-level super user with full system access	User & role management, system config
Property Manager	Manages one or more properties in the portfolio	Leases, billing, maintenance, reports
Tenant	Resident or business occupying a unit	Pay rent, raise complaints, view notices
Maintenance Staff	Technicians assigned to repair work orders	Update ticket status, log work done

1.3 Key Features

- Multi-role authentication and JWT-based authorization
- Property, building, and unit lifecycle management
- Full lease contract management with auto-renewal hooks
- Automated invoice generation and rent payment tracking
- Maintenance request and ticket workflow with SLA tracking
- Real-time event-driven notifications via ActiveMQ
- Comprehensive reporting (occupancy, revenue, aging AR)
- Full audit trail on all domain mutations

1.4 Assumptions & Constraints

Category	Detail
Deployment	Single-region deployment initially; must support horizontal scaling via stateless services
Currency	Single-currency (configurable per tenant); no multi-currency FX in V1
Payments	V1 tracks manual payments; gateway integration (Stripe/PayPal) is a V2 feature
Communication	Notifications via in-app + email (SMTP); SMS is a V2 feature
Data Volume	Designed for up to 10,000 units and 50,000 users per installation

2. Functional Requirements

2.1 User Management

Use Cases

- Admin registers new users and assigns roles
- Users authenticate via username/password; receive JWT
- Password reset via email token flow
- Admin can deactivate/reactivate accounts
- Role hierarchy: ADMIN > PROPERTY_MANAGER > MAINTENANCE_STAFF > TENANT

User Stories

US-001: As an Admin, I want to create a user account and assign a role so that the person gains appropriate system access.

US-002: As a Tenant, I want to log in securely and view only my own lease and payment data.

US-003: As any User, I want to reset my password via a time-limited email link.

Workflow: Login & JWT Issuance

```
Client → POST /api/auth/login { username, password }
AuthService → validates credentials → generates JWT (access 15m) + Refresh Token
(7d)
JWT payload: { sub: userId, roles: [...], propertyIds: [...], iat, exp }
Client stores JWT; sends as Authorization: Bearer <token> on every request
On expiry → POST /api/auth/refresh { refreshToken } → new JWT pair
```

2.2 Property Management

Use Cases

- Manager creates a Property (e.g., "Palm Hills Compound")
- Manager adds Buildings under a Property
- Manager adds Units under a Building with type, floor, area, and rent amount
- Admin can transfer a property to a different manager
- View occupancy rate per property/building

User Stories

US-010: As a Property Manager, I want to add buildings and units so I can track the full physical hierarchy of my portfolio.

US-011: As an Admin, I want to view a dashboard of all properties with their occupancy rates.

Hierarchy

```
Property → 1:N → Building → 1:N → Unit
```

2.3 Lease Management

Use Cases

- Manager creates a lease linking a Tenant to a Unit
- Lease has start date, end date, monthly rent, security deposit, grace period

- System auto-generates monthly invoices when lease is ACTIVE
- Manager can renew or terminate a lease
- Lease status transitions: DRAFT → ACTIVE → EXPIRED / TERMINATED

User Stories

US-020: As a Manager, I want to create a lease for a tenant so that the unit is marked occupied and billing begins.

US-021: As a Tenant, I want to see my current lease terms, rent amount, and expiry date.

Workflow: Lease Activation

```
Manager → POST /api/leases { tenantId, unitId, startDate, endDate, monthlyRent,
deposit }
LeaseService validates: unit must be VACANT, tenant must exist
Lease saved with status=DRAFT
Manager → PATCH /api/leases/{id}/activate
Unit status updated to OCCUPIED
Event published: LeaseCreated → BillingService generates first invoice
```

2.4 Payments & Billing

Use Cases

- System generates monthly invoices automatically per active lease
- Tenant views outstanding invoices
- Manager records a payment against an invoice
- System calculates late fees after grace period
- Invoices have statuses: PENDING, PARTIALLY_PAID, PAID, OVERDUE, VOID
- Manager can generate statements for a date range

User Stories

US-030: As a Tenant, I want to see all my invoices and their due dates so I can pay on time.

US-031: As a Manager, I want to record a cash/bank-transfer payment so the invoice is marked paid.

Workflow: Invoice & Payment

```
Scheduled Job: runs 1st of each month
  → Queries all ACTIVE leases
  → Generates Invoice { leaseId, amount, dueDate = lease.paymentDueDay,
status=PENDING }

Manager → POST /api/payments { invoiceId, amount, method, referenceNo }
PaymentService:
  → Creates Payment record
  → Recalculates invoice.paidAmount
  → Updates invoice status (PARTIALLY_PAID / PAID)
  → Publishes PaymentCompleted event → NotificationService
```

2.5 Maintenance Requests

Use Cases

- Tenant submits a maintenance request with category, description, photos
- Manager reviews and assigns to Maintenance Staff

- Staff updates progress; marks as RESOLVED
- Ticket statuses: OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED
- Manager can escalate or reopen tickets
- SLA tracking: target resolution time per priority level

User Stories

US-040: As a Tenant, I want to report a broken AC unit so a technician can be dispatched.

US-041: As Maintenance Staff, I want to update ticket progress so the manager can track status in real time.

Priority & SLA Matrix

Priority	Example	Target Resolution	Escalation After
CRITICAL	Flood, gas leak, fire safety	4 hours	2 hours
HIGH	No hot water, heating failure	24 hours	12 hours
MEDIUM	Broken appliance, leaky faucet	72 hours	48 hours
LOW	Paint, cosmetic issues	7 days	5 days

2.6 Notifications

Use Cases

- Tenants receive rent-due reminders 7 and 3 days before due date
- Tenants notified when maintenance ticket status changes
- Managers notified of new maintenance requests
- Automatic lease expiry warnings 60/30/7 days before end
- Notification channels: In-App (database) + Email (SMTP)

Notification Types

Event	Recipient	Channel	Timing
Rent Due Reminder	Tenant	Email + In-App	T-7, T-3 days
Payment Received	Tenant	Email + In-App	Immediate
Maintenance Update	Tenant	Email + In-App	On status change
New Maintenance Request	Manager	Email + In-App	Immediate
Lease Expiry Warning	Tenant + Manager	Email + In-App	T-60, T-30, T-7

2.7 Reporting

Report Types

Report	Description	Access Role
Occupancy Report	Vacancy/occupancy rate by property, building, or unit	Admin, Manager

Revenue Report	Expected vs collected rent by date range	Admin, Manager
Aging AR Report	Outstanding invoices grouped by 0-30, 31-60, 61-90, 90+ days	Admin, Manager
Maintenance Report	Open/closed tickets by property, category, or date	Admin, Manager
Lease Expiry Report	Leases expiring within configurable window	Admin, Manager

3. Non-Functional Requirements

3.1 Scalability

- Stateless REST services — horizontal scaling via load balancer (NGINX/AWS ALB)
- Database connection pooling via HikariCP (max-pool-size: 20 per node)
- Read replicas for reporting queries; writer for mutations
- Asynchronous invoice generation and notification dispatch via message queues
- Pagination enforced on all list endpoints (default page size: 20, max: 100)

3.2 Security

- JWT RS256-signed tokens (asymmetric keys); access token TTL: 15 minutes
- Refresh token stored server-side in DB with hash; single-use rotation
- RBAC enforced at method level via @PreAuthorize annotations
- Input validation via Bean Validation (JSR-380) on all DTOs
- SQL injection prevention via JPA parameterized queries only
- HTTPS enforced; HSTS header with 1-year max-age
- Sensitive config via environment variables / AWS Secrets Manager
- Rate limiting: 100 req/min per IP on public endpoints

3.3 Performance

Metric	Target	Measurement Method
API P95 Latency	< 300ms	Micrometer + Prometheus scraping
DB Query Time (common)	< 50ms	Hibernate slow-query log threshold
Invoice Batch Generation	10,000 invoices < 2 min	Scheduled job duration log
Concurrent Users	500 concurrent sessions	JMeter load test at 500 VU

3.4 Availability

- Target: 99.9% uptime (< 8.7 hours downtime/year)
- Zero-downtime deployments via rolling updates (Kubernetes/Blue-Green)
- Database daily backups + point-in-time recovery (PITR) enabled
- ActiveMQ durable queues to prevent message loss during restarts
- Health check endpoint: GET /actuator/health for load-balancer probing

3.5 Audit & Logging

- All entity mutations tracked with: created_by, created_at, updated_by, updated_at
- AuditLog table records: entity_type, entity_id, action, old_value (JSON), new_value (JSON), performed_by, timestamp
- Structured JSON logs via Logback + Logstash encoder
- Correlation ID injected into MDC for end-to-end request tracing
- Sensitive field masking (passwords, tokens) in all logs

4. High-Level Architecture

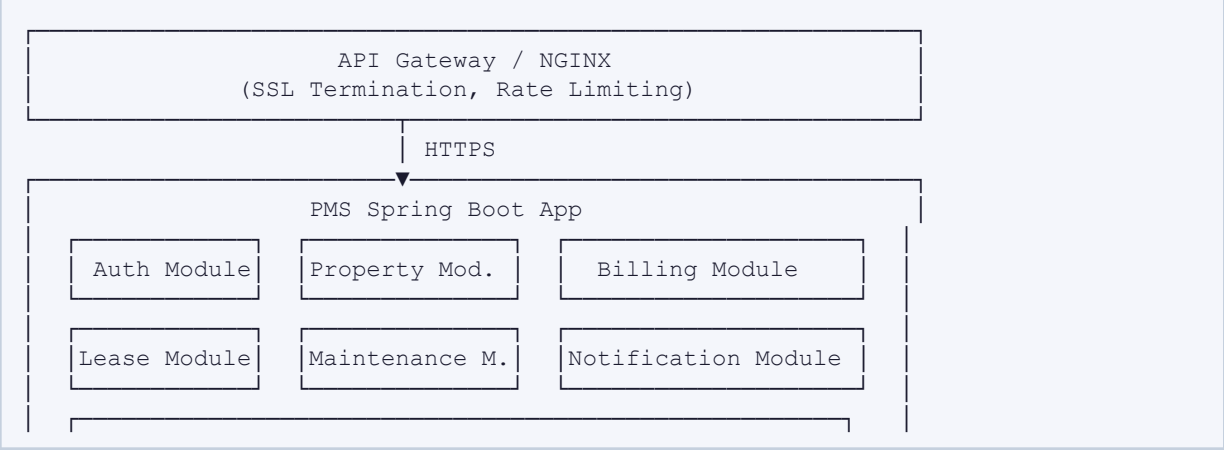
4.1 Architecture Decision: Modular Monolith

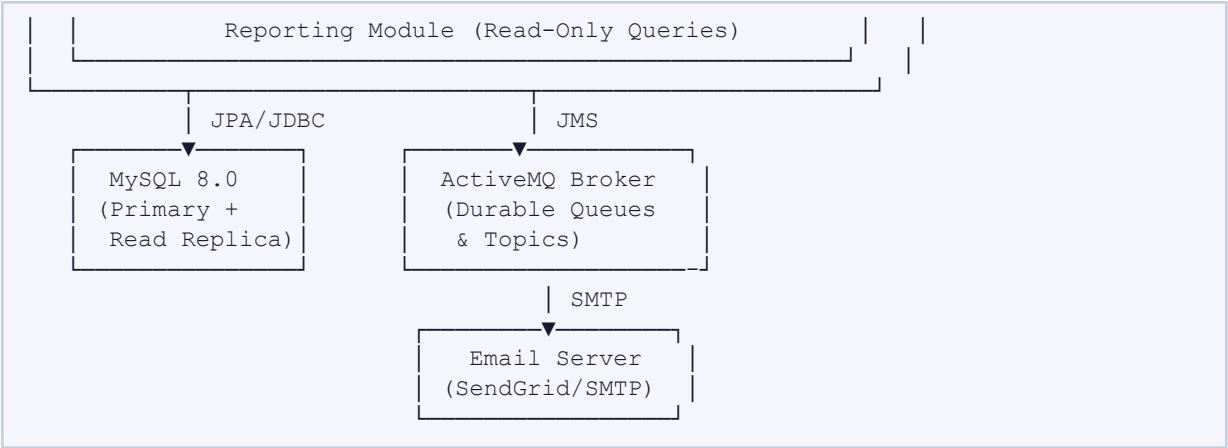
📌 Decision: We adopt a Modular Monolith for V1, not a distributed microservices architecture. At medium scale (≤50k users, ≤10k units), the operational overhead of microservices (service mesh, distributed tracing, saga orchestration) outweighs the benefits. The codebase is structured with clear module boundaries so migration to microservices in V2 is low-friction.

4.2 Layered Architecture

Layer	Components	Responsibility
Presentation	@RestController, DTOs, GlobalExceptionHandler	HTTP in/out, validation, serialization
Application	@Service, @Transactional, Event Publishers	Business logic, transaction coordination
Domain	@Entity, Domain Enums, Value Objects	Core domain model and invariants
Infrastructure	@Repository, JPA, ActiveMQ, SMTP, Scheduler	External system integrations

4.3 Component Diagram (Textual)





4.4 Data Flow: Rent Payment

```
1. Tenant  → POST /api/payments  (JWT Auth)
2. PaymentController → validates DTO (BeanValidation)
3. PaymentService.recordPayment():
   a. Load Invoice by id (must belong to tenant's lease)
   b. Create Payment record, update Invoice.paidAmount
   c. Determine new InvoiceStatus (PARTIALLY_PAID | PAID)
   d. Save via @Transactional
   e. Publish PaymentCompletedEvent to queue: billing.payment
4. NotificationConsumer receives event:
   a. Sends email receipt to tenant
   b. Creates in-app Notification record
5. ReportingService reads audit logs for revenue aggregation
```

5. Database Design

5.1 Entity List

Entity	Description
users	All system users regardless of role
roles	System roles: ADMIN, PROPERTY_MANAGER, MAINTENANCE_STAFF, TENANT
user_roles	Junction table for N:M user ↔ role assignment
refresh_tokens	Server-side refresh token store (hashed)
properties	Top-level property entity (e.g. compound, complex)
buildings	Physical building within a property
units	Individual rentable unit within a building
leases	Contract linking a tenant to a unit for a date range
invoices	Monthly billing record generated per active lease

payments	Actual payment recorded against an invoice
maintenance_requests	Tenant-submitted repair/issue requests
maintenance_assignments	Assignment of staff to a maintenance request
notifications	In-app notification records per user
audit_logs	Immutable audit trail of all domain mutations

5.2 Full Entity Relationship Diagram (Text-Based ERD)

```
users ||--o{ user_roles         : "assigned"
roles ||--o{ user_roles         : "defines"
users ||--o{ refresh_tokens     : "owns"
users ||--o{ properties         : "manages (manager_id)"
properties ||--o{ buildings     : "contains"
buildings ||--o{ units          : "contains"
users ||--o{ leases             : "tenant_id"
units ||--o{ leases             : "occupies"
leases ||--o{ invoices          : "generates"
invoices ||--o{ payments        : "settled by"
units ||--o{ maintenance_requests : "raised for"
users ||--o{ maintenance_requests : "raised by (tenant)"
maintenance_requests ||--o{ maintenance_assignments : "assigned via"
users ||--o{ maintenance_assignments : "assigned to (staff)"
users ||--o{ notifications      : "receives"
```

5.3 Table Definitions

Table: users

```
CREATE TABLE users (
  id            BIGINT          NOT NULL AUTO_INCREMENT,
  uuid          CHAR(36)        NOT NULL UNIQUE,
  username      VARCHAR(100)    NOT NULL UNIQUE,
  email         VARCHAR(255)    NOT NULL UNIQUE,
  password_hash VARCHAR(255)    NOT NULL,
  full_name     VARCHAR(200)    NOT NULL,
  phone         VARCHAR(30),
  status        ENUM('ACTIVE','INACTIVE','SUSPENDED') NOT NULL DEFAULT 'ACTIVE',
  created_at    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by    BIGINT,
  updated_by    BIGINT,
  PRIMARY KEY (id)
);
```

Table: roles

```
CREATE TABLE roles (
  id          BIGINT       NOT NULL AUTO_INCREMENT,
```

```
  name        VARCHAR(50)  NOT NULL UNIQUE,   -- ADMIN, PROPERTY_MANAGER, etc.
  description VARCHAR(255),
  PRIMARY KEY (id)
);

CREATE TABLE user_roles (
  user_id  BIGINT NOT NULL,
  role_id  BIGINT NOT NULL,
  PRIMARY KEY (user_id, role_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
);
```

Table: refresh_tokens

```
CREATE TABLE refresh_tokens (
  id           BIGINT       NOT NULL AUTO_INCREMENT,
  user_id      BIGINT       NOT NULL,
  token_hash   VARCHAR(255) NOT NULL UNIQUE,
  expires_at   DATETIME     NOT NULL,
  revoked      BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Table: properties

```
CREATE TABLE properties (
  id          BIGINT       NOT NULL AUTO_INCREMENT,
  name        VARCHAR(200) NOT NULL,
  address     VARCHAR(500) NOT NULL,
  city        VARCHAR(100) NOT NULL,
  country     VARCHAR(100) NOT NULL DEFAULT 'Egypt',
  manager_id  BIGINT       NOT NULL,
  status      ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by  BIGINT,
  updated_by  BIGINT,
  PRIMARY KEY (id),
  FOREIGN KEY (manager_id) REFERENCES users(id)
);
```

Table: buildings

```
CREATE TABLE buildings (
  id           BIGINT       NOT NULL AUTO_INCREMENT,
  property_id  BIGINT       NOT NULL,
  name         VARCHAR(200) NOT NULL,
  floors       INT          NOT NULL DEFAULT 1,
  year_built   YEAR,
  status       ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
  created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by   BIGINT,
```

```
  updated_by   BIGINT,
  PRIMARY KEY (id),
  FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
);
```

Table: units

```
CREATE TABLE units (
  id             BIGINT         NOT NULL AUTO_INCREMENT,
  building_id    BIGINT         NOT NULL,
  unit_number    VARCHAR(20)    NOT NULL,
  floor          INT            NOT NULL,
  unit_type      ENUM('STUDIO','ONE_BR','TWO_BR','THREE_BR','COMMERCIAL') NOT
NULL,
  area_sqm       DECIMAL(8,2)   NOT NULL,
  base_rent      DECIMAL(12,2)  NOT NULL,
  status         ENUM('VACANT','OCCUPIED','MAINTENANCE','RESERVED') NOT NULL
DEFAULT 'VACANT',
  notes          TEXT,
  created_at     DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by     BIGINT,
  updated_by     BIGINT,
  PRIMARY KEY (id),
  UNIQUE KEY uk_building_unit (building_id, unit_number),
  FOREIGN KEY (building_id) REFERENCES buildings(id) ON DELETE CASCADE
);
```

Table: leases

```
CREATE TABLE leases (
  id               BIGINT        NOT NULL AUTO_INCREMENT,
  unit_id          BIGINT        NOT NULL,
  tenant_id        BIGINT        NOT NULL,
  start_date       DATE          NOT NULL,
  end_date         DATE          NOT NULL,
  monthly_rent     DECIMAL(12,2) NOT NULL,
  security_deposit DECIMAL(12,2) NOT NULL DEFAULT 0,
  payment_due_day  TINYINT       NOT NULL DEFAULT 1,     -- day of month (1-28)
  grace_period_days TINYINT      NOT NULL DEFAULT 5,
  late_fee_pct     DECIMAL(5,2)  NOT NULL DEFAULT 0,
  status           ENUM('DRAFT','ACTIVE','EXPIRED','TERMINATED') NOT NULL DEFAULT
'DRAFT',
  terminated_at    DATE,
  termination_reason TEXT,
  created_at       DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at       DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by       BIGINT,
  updated_by       BIGINT,
  PRIMARY KEY (id),
  FOREIGN KEY (unit_id)   REFERENCES units(id),
  FOREIGN KEY (tenant_id) REFERENCES users(id),
  INDEX idx_lease_tenant   (tenant_id),
  INDEX idx_lease_unit     (unit_id),
  INDEX idx_lease_status   (status),
  INDEX idx_lease_end_date (end_date)
);
```

Table: invoices

```
CREATE TABLE invoices (
  id             BIGINT        NOT NULL AUTO_INCREMENT,
  invoice_number VARCHAR(30)   NOT NULL UNIQUE,   -- e.g. INV-2024-001234
  lease_id       BIGINT        NOT NULL,
  billing_month  DATE          NOT NULL,           -- first day of billing month
  due_date       DATE          NOT NULL,
  amount         DECIMAL(12,2) NOT NULL,
  late_fee       DECIMAL(12,2) NOT NULL DEFAULT 0,
  paid_amount    DECIMAL(12,2) NOT NULL DEFAULT 0,
  status         ENUM('PENDING','PARTIALLY_PAID','PAID','OVERDUE','VOID') NOT NULL
DEFAULT 'PENDING',
  notes          TEXT,
  created_at     DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by     BIGINT,
  PRIMARY KEY (id),
  FOREIGN KEY (lease_id) REFERENCES leases(id),
  INDEX idx_invoice_lease  (lease_id),
  INDEX idx_invoice_status (status),
  INDEX idx_invoice_due    (due_date)
);
```

Table: payments

```
CREATE TABLE payments (
  id             BIGINT        NOT NULL AUTO_INCREMENT,
  invoice_id     BIGINT        NOT NULL,
  amount         DECIMAL(12,2) NOT NULL,
  payment_date   DATE          NOT NULL,
  method         ENUM('CASH','BANK_TRANSFER','CHEQUE','ONLINE') NOT NULL,
  reference_no   VARCHAR(100),
  notes          TEXT,
  recorded_by    BIGINT        NOT NULL,           -- manager who recorded it
  created_at     DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  FOREIGN KEY (invoice_id)   REFERENCES invoices(id),
  FOREIGN KEY (recorded_by)  REFERENCES users(id)
);
```

Table: maintenance_requests

```
CREATE TABLE maintenance_requests (
  id             BIGINT       NOT NULL AUTO_INCREMENT,
  unit_id        BIGINT       NOT NULL,
  raised_by      BIGINT       NOT NULL,           -- tenant user_id
  category
ENUM('PLUMBING','ELECTRICAL','HVAC','STRUCTURAL','APPLIANCE','OTHER') NOT NULL,
  priority       ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'MEDIUM',
  title          VARCHAR(300) NOT NULL,
  description    TEXT         NOT NULL,
  status
ENUM('OPEN','ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED','CANCELLED') NOT NULL
DEFAULT 'OPEN',
  resolved_at    DATETIME,
  resolution_notes TEXT,
  created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  updated_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  created_by     BIGINT,
  updated_by     BIGINT,
  PRIMARY KEY (id),
  FOREIGN KEY (unit_id)    REFERENCES units(id),
  FOREIGN KEY (raised_by)  REFERENCES users(id)
);
```

Table: maintenance_assignments

```
CREATE TABLE maintenance_assignments (
  id             BIGINT   NOT NULL AUTO_INCREMENT,
  request_id     BIGINT   NOT NULL,
  staff_id       BIGINT   NOT NULL,
  assigned_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  assigned_by    BIGINT   NOT NULL,
  notes          TEXT,
  PRIMARY KEY (id),
  FOREIGN KEY (request_id) REFERENCES maintenance_requests(id),
  FOREIGN KEY (staff_id)   REFERENCES users(id),
  FOREIGN KEY (assigned_by) REFERENCES users(id)
);
```

Table: notifications

```
CREATE TABLE notifications (
  id          BIGINT       NOT NULL AUTO_INCREMENT,
  user_id     BIGINT       NOT NULL,
  type        VARCHAR(80)  NOT NULL,       -- RENT_DUE, TICKET_UPDATED, etc.
  title       VARCHAR(300) NOT NULL,
  body        TEXT         NOT NULL,
  entity_type VARCHAR(80),                 -- INVOICE, MAINTENANCE_REQUEST, etc.
  entity_id   BIGINT,
  is_read     BOOLEAN      NOT NULL DEFAULT FALSE,
  sent_email  BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_notif_user_read (user_id, is_read)
);
```

Table: audit_logs

```
CREATE TABLE audit_logs (
  id           BIGINT       NOT NULL AUTO_INCREMENT,
  entity_type  VARCHAR(80)  NOT NULL,
  entity_id    BIGINT       NOT NULL,
  action       ENUM('CREATE','UPDATE','DELETE','STATUS_CHANGE') NOT NULL,
  old_value    JSON,
  new_value    JSON,
  performed_by BIGINT       NOT NULL,
  performed_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ip_address   VARCHAR(45),
  PRIMARY KEY (id),
  INDEX idx_audit_entity (entity_type, entity_id),
  INDEX idx_audit_user   (performed_by)
);
```

5.4 Relationship Summary

Relationship	Type	FK Column	Cascade
users → properties	1:N	properties.manager_id	Restrict
properties → buildings	1:N	buildings.property_id	Cascade Delete
buildings → units	1:N	units.building_id	Cascade Delete
units → leases	1:N	leases.unit_id	Restrict
users → leases	1:N	leases.tenant_id	Restrict
leases → invoices	1:N	invoices.lease_id	Restrict
invoices → payments	1:N	payments.invoice_id	Restrict
users ↔ roles	N:M	user_roles (junction)	Cascade Delete

6. API Design

6.1 Authentication

Method	Endpoint	Description	Auth	Status
POST	/api/auth/login	Authenticate; returns JWT pair	None	200, 401
POST	/api/auth/refresh	Refresh access token	None	200, 403
POST	/api/auth/logout	Revoke refresh token	JWT	204
POST	/api/auth/password-reset/request	Send reset email	None	202

Login Request / Response

```
// POST /api/auth/login
{
  "username": "ahmed.hassan",
  "password": "S3cur3P@ssword!"
}

// 200 OK
{
  "accessToken":  "eyJhbGciOiJSUzI1NiJ9...",
  "refreshToken": "8f4a2c1d-...",
  "expiresIn":    900,
  "user": {
    "id": 42,
    "username": "ahmed.hassan",
    "email": "ahmed@example.com",
    "roles": ["PROPERTY_MANAGER"],
    "fullName": "Ahmed Hassan"
```

```
  }
}
```

6.2 Property & Unit Endpoints

Method	Endpoint	Description	Role
GET	/api/properties	List all properties (paginated)	ADMIN, MANAGER
POST	/api/properties	Create property	ADMIN
GET	/api/properties/{id}	Get property detail	ADMIN, MANAGER
PUT	/api/properties/{id}	Update property	ADMIN, MANAGER
GET	/api/properties/{id}/buildings	List buildings in property	ADMIN, MANAGER
POST	/api/buildings/{id}/units	Add unit to building	ADMIN, MANAGER
GET	/api/units/{id}	Get unit details	ADMIN, MANAGER
PATCH	/api/units/{id}/status	Update unit status	ADMIN, MANAGER

6.3 Lease Endpoints

Method	Endpoint	Description	Role
POST	/api/leases	Create new lease (status=DRAFT)	MANAGER
PATCH	/api/leases/{id}/activate	Activate lease; generates first invoice	MANAGER
GET	/api/leases/{id}	Get lease details	MANAGER, TENANT (own)
PATCH	/api/leases/{id}/terminate	Terminate lease with reason	MANAGER
GET	/api/tenants/me/lease	Get current tenant's active lease	TENANT

Create Lease Request / Response

```
// POST /api/leases
{
  "unitId":           101,
  "tenantId":         55,
  "startDate":        "2024-02-01",
  "endDate":          "2025-01-31",
  "monthlyRent":      8500.00,
  "securityDeposit":  17000.00,
  "paymentDueDay":    1,
  "gracePeriodDays":  5
```

```
}

// 201 Created
{
  "id":              78,
  "status":          "DRAFT",
  "unit":            { "id": 101, "unitNumber": "3B", "building": "Tower A" },
  "tenant":          { "id": 55,  "fullName": "Sara Ahmed", "email":
"sara@example.com" },
  "startDate":       "2024-02-01",
  "endDate":         "2025-01-31",
  "monthlyRent":     8500.00,
  "createdAt":       "2024-01-15T10:30:00Z"
}
```

6.4 Payment Endpoints

Method	Endpoint	Description	Role
GET	/api/invoices?leaseId={id}	List invoices for a lease	MANAGER, TENANT (own)
GET	/api/invoices/{id}	Get invoice detail	MANAGER, TENANT (own)
POST	/api/payments	Record a payment against invoice	MANAGER
GET	/api/invoices/{id}/payments	List payments on invoice	MANAGER
PATCH	/api/invoices/{id}/void	Void invoice (no payments made)	ADMIN, MANAGER

6.5 Maintenance Endpoints

Method	Endpoint	Description	Role
POST	/api/maintenance-requests	Submit a new request	TENANT
GET	/api/maintenance-requests	List requests (filtered by property/status)	MANAGER
PATCH	/api/maintenance-requests/{id}/assign	Assign to staff member	MANAGER
PATCH	/api/maintenance-requests/{id}/status	Update status (IN_PROGRESS, RESOLVED)	STAFF, MANAGER
GET	/api/tenants/me/maintenance-requests	Tenant's own requests	TENANT

6.6 Standard Error Response

```
// 400 Bad Request — validation failure
{
  "timestamp": "2024-01-15T10:30:00Z",
  "status": 400,
  "error": "Validation Failed",
  "message": "Request body contains invalid fields",
  "correlationId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "errors": [
    { "field": "endDate",  "message": "must be after startDate" },
    { "field": "monthlyRent", "message": "must be greater than 0" }
  ]
}

// 403 Forbidden — insufficient role
{
  "timestamp": "2024-01-15T10:30:00Z",
  "status": 403,
  "error": "Access Denied",
  "message": "You do not have permission to access this resource",
  "correlationId": "a1b2c3d4-..."
}
```

7. Event-Driven Design

7.1 ActiveMQ Topology

```
Queues (Point-to-Point — guaranteed single consumer):
  billing.invoice.generate       — triggers invoice creation job
  notification.email.outbound    — email sending queue

Topics (Pub/Sub — all subscribers receive event):
  pms.lease.created              — subscribed by: Billing, Notification
  pms.payment.completed          — subscribed by: Notification, Reporting
  pms.maintenance.requested      — subscribed by: Notification
  pms.maintenance.status.changed — subscribed by: Notification
  pms.lease.expiring             — subscribed by: Notification
```

7.2 Event Definitions

LeaseCreatedEvent

```
// Topic: pms.lease.created
{
  "eventId":    "uuid-v4",
  "eventType":  "LEASE_CREATED",
  "timestamp":  "2024-01-15T10:30:00Z",
  "payload": {
    "leaseId":     78,
    "unitId":      101,
    "tenantId":    55,
    "managerId":   12,
    "startDate":   "2024-02-01",
    "endDate":     "2025-01-31",
    "monthlyRent": 8500.00
```

```
  }
}
// Consumers: BillingService (generates first invoice), NotificationService
(welcome email)
```

PaymentCompletedEvent

```
// Topic: pms.payment.completed
{
  "eventId":    "uuid-v4",
  "eventType":  "PAYMENT_COMPLETED",
  "timestamp":  "2024-01-15T11:00:00Z",
  "payload": {
    "paymentId":    220,
    "invoiceId":    189,
    "leaseId":      78,
    "tenantId":     55,
    "amount":       8500.00,
    "method":       "BANK_TRANSFER",
    "invoiceStatus":"PAID"
  }
}
// Consumers: NotificationService (receipt email), ReportingService (revenue
update)
```

MaintenanceRequestedEvent

```
// Topic: pms.maintenance.requested
{
  "eventId":    "uuid-v4",
  "eventType":  "MAINTENANCE_REQUESTED",
  "timestamp":  "2024-01-15T14:00:00Z",
  "payload": {
    "requestId":  55,
    "unitId":     101,
    "tenantId":   55,
    "managerId":  12,
    "category":   "PLUMBING",
    "priority":   "HIGH",
    "title":      "Water heater not working"
  }
}
// Consumers: NotificationService (alerts manager)
```

MaintenanceStatusChangedEvent

```
// Topic: pms.maintenance.status.changed
{
  "eventId":   "uuid-v4",
  "eventType": "MAINTENANCE_STATUS_CHANGED",
  "timestamp": "2024-01-16T09:00:00Z",
  "payload": {
    "requestId":   55,
    "tenantId":    55,
    "oldStatus":   "ASSIGNED",
    "newStatus":   "IN_PROGRESS",
    "updatedBy":   33,
    "notes":       "Parts ordered; work starts tomorrow"
```

```
  }
}
// Consumers: NotificationService (status update to tenant)
```

LeaseExpiringEvent

```
// Topic: pms.lease.expiring
// Published by: scheduled job running daily
{
  "eventId":   "uuid-v4",
  "eventType": "LEASE_EXPIRING",
  "timestamp": "2024-01-15T00:05:00Z",
  "payload": {
    "leaseId":      78,
    "tenantId":     55,
    "managerId":    12,
    "endDate":      "2024-02-14",
    "daysRemaining": 30
  }
}
// Consumers: NotificationService (expiry warning to both parties)
```

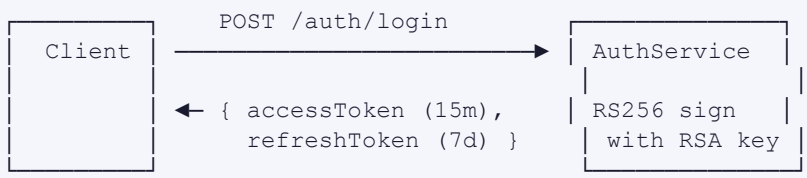
7.3 Event Producer & Consumer Pattern

```
// Producer (in Service layer)
@Service
public class LeaseService {
    @Autowired private JmsTemplate jmsTemplate;
    @Autowired private ObjectMapper objectMapper;

    @Transactional
    public LeaseDto activateLease(Long leaseId) {
        Lease lease = leaseRepository.findById(leaseId).orElseThrow();
        lease.setStatus(LeaseStatus.ACTIVE);
        leaseRepository.save(lease);

        LeaseCreatedEvent event = buildEvent(lease);
        jmsTemplate.convertAndSend("pms.lease.created",
objectMapper.writeValueAsString(event));
        return leaseMapper.toDto(lease);
    }
}

// Consumer (in Notification module)
@Component
public class LeaseEventConsumer {
    @JmsListener(destination = "pms.lease.created",
                 containerFactory = "topicListenerFactory")
    public void onLeaseCreated(String message) {
        LeaseCreatedEvent event = objectMapper.readValue(message,
LeaseCreatedEvent.class);
        notificationService.sendWelcomeEmail(event.getPayload().getTenantId());

notificationService.createInAppNotification(event.getPayload().getTenantId(),
            "LEASE_ACTIVATED", "Your lease has been activated successfully.");
    }
```

```
}
```

8. Security Design

8.1 JWT Authentication Flow




```
On every request:
Authorization: Bearer <accessToken>
↓
JwtAuthFilter → verifies signature (public key) → extracts claims
↓
SecurityContext populated → @PreAuthorize checks role

On token expiry:
POST /auth/refresh { refreshToken }
→ Server verifies hash in DB → issues new token pair → rotates refresh token
```

8.2 Role-Based Access Control (RBAC) Matrix

Resource	ADMIN	MANAGER	STAFF	TENANT	Notes
User Management	✅ Full	❌	❌	❌	Admin only
Properties CRUD	✅ Full	✅ Own	❌	❌	Manager: own properties
Leases: Create	✅	✅	❌	❌	
Leases: View	✅ All	✅ Own	❌	✅ Own	Tenant: own lease
Payments: Record	✅	✅	❌	❌	
Maintenance: Submit	✅	✅	❌	✅	Tenant submits
Maintenance: Assign	✅	✅	❌	❌	
Maintenance: Update	✅	✅	✅ Assigned	❌	Staff: assigned only
Reports	✅ All	✅ Own	❌	❌	

8.3 Data Protection Strategies

- Passwords: bcrypt with cost factor 12 (never stored in plaintext or logged)
- JWTs: RS256 asymmetric signing; public key exposed at /.well-known/jwks.json
- Refresh tokens: stored as SHA-256 hash; original returned to client only once
- Database: encrypted at rest (AES-256); TLS in transit between app and DB
- Input sanitization: all user-facing text fields sanitized to prevent XSS
- CORS: configured whitelist of allowed origins; no wildcard in production
- Tenant data isolation: all queries filtered by authenticated user's scope
- API rate limiting: Bucket4j per user + per IP

9. Liquibase Design

9.1 Versioning Strategy

- One changelog file per feature/sprint: db/changelog/v1.0.0/
- Files named: YYYYMMDD-NNN-description.xml
- Master changelog (db/changelog/db.changelog-master.xml) includes all versions
- Never modify existing changesets; create new ones for corrections
- Rollback scripts defined alongside each changeset

9.2 Master Changelog

```
<!-- db/changelog/db.changelog-master.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
    xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
        http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.9.xsd">

    <include file="db/changelog/v1.0.0/20240101-001-create-users.xml"/>
    <include file="db/changelog/v1.0.0/20240101-002-create-roles.xml"/>
    <include file="db/changelog/v1.0.0/20240101-003-create-properties.xml"/>
    <include file="db/changelog/v1.0.0/20240101-004-create-units.xml"/>
    <include file="db/changelog/v1.0.0/20240101-005-create-leases.xml"/>
    <include file="db/changelog/v1.0.0/20240101-006-create-billing.xml"/>
    <include file="db/changelog/v1.0.0/20240101-007-create-maintenance.xml"/>
    <include file="db/changelog/v1.0.0/20240101-008-create-notifications.xml"/>
    <include file="db/changelog/v1.0.0/20240101-009-seed-roles.xml"/>

</databaseChangeLog>
```

9.3 Sample Changeset: Create Users Table

```
<!-- db/changelog/v1.0.0/20240101-001-create-users.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="...">

    <changeSet id="20240101-001" author="pms-dev">
```

```
        <createTable tableName="users">
            <column name="id" type="BIGINT" autoIncrement="true">
                <constraints primaryKey="true" nullable="false"/>
            </column>
            <column name="uuid" type="CHAR(36)">
                <constraints unique="true" nullable="false"/>
            </column>
            <column name="username" type="VARCHAR(100)">
                <constraints unique="true" nullable="false"/>
            </column>
            <column name="email" type="VARCHAR(255)">
                <constraints unique="true" nullable="false"/>
            </column>
            <column name="password_hash" type="VARCHAR(255)">
                <constraints nullable="false"/>
            </column>
            <column name="full_name" type="VARCHAR(200)">
                <constraints nullable="false"/>
            </column>
            <column name="phone" type="VARCHAR(30)"/>
            <column name="status" type="ENUM('ACTIVE','INACTIVE','SUSPENDED')"
                defaultValue="ACTIVE">
                <constraints nullable="false"/>
            </column>
            <column name="created_at" type="DATETIME"
                defaultValueComputed="CURRENT_TIMESTAMP">
                <constraints nullable="false"/>
            </column>
            <column name="updated_at" type="DATETIME"
                defaultValueComputed="CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP">
                <constraints nullable="false"/>
            </column>
            <column name="created_by" type="BIGINT"/>
            <column name="updated_by" type="BIGINT"/>
        </createTable>

        <rollback>
            <dropTable tableName="users"/>
        </rollback>
    </changeSet>

</databaseChangeLog>
```

9.4 Sample Changeset: Seed Default Roles

```
<!-- 20240101-009-seed-roles.xml -->
    <changeSet id="20240101-009" author="pms-dev">
        <insert tableName="roles">
            <column name="name" value="ADMIN"/>
            <column name="description" value="Full system administrator"/>
        </insert>
        <insert tableName="roles">
            <column name="name" value="PROPERTY_MANAGER"/>
            <column name="description" value="Manages assigned properties"/>
        </insert>
        <insert tableName="roles">
            <column name="name" value="MAINTENANCE_STAFF"/>
```

```
            <column name="description" value="Handles maintenance work orders"/>
        </insert>
        <insert tableName="roles">
            <column name="name" value="TENANT"/>
            <column name="description" value="Residential or commercial tenant"/>
        </insert>
    </changeSet>
```

9.5 application.yml Liquibase Config

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/pms_db?useSSL=true&serverTimezone=UTC
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20
      connection-timeout: 30000
  jpa:
    hibernate:
      ddl-auto: validate   # Liquibase owns the schema; Hibernate validates only
    show-sql: false
    properties:
      hibernate.format_sql: true
      hibernate.slow_query_threshold: 1000  # ms
  liquibase:
    change-log: classpath:db/changelog/db.changelog-master.xml
    enabled: true
```

10. Spring Boot Folder Structure

```
pms-backend/
├── src/main/java/com/pms/
│   ├── PmsApplication.java
│   │
│   ├── config/
│   │   ├── SecurityConfig.java          # Spring Security + JWT filter chain
│   │   ├── JwtConfig.java               # RSA key loading, JWT builder/parser
│   │   ├── ActiveMQConfig.java          # ConnectionFactory, JmsTemplate,
factories
│   │   ├── SchedulerConfig.java         # @EnableScheduling
│   │   └── AuditConfig.java             # SpringDataAuditing + AuditorAware
│   │
│   ├── security/
│   │   ├── JwtAuthFilter.java           # OncePerRequestFilter — token validation
│   │   ├── CustomUserDetailsService.java
│   │   ├── UserPrincipal.java
│   │   └── RateLimitFilter.java         # Bucket4j per-user rate limit
│   │
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── controller/AuthController.java
│   │   │   ├── dto/LoginRequest.java, LoginResponse.java, RefreshRequest.java
│   │   │   └── service/AuthService.java
│   │   │
│   │   ├── user/
```

```
│   │   │   ├── controller/UserController.java
│   │   │   ├── dto/CreateUserRequest.java, UserDto.java
│   │   │   ├── entity/User.java, Role.java, RefreshToken.java
│   │   │   ├── mapper/UserMapper.java   # MapStruct
│   │   │   ├── repository/UserRepository.java, RoleRepository.java
│   │   │   └── service/UserService.java
│   │   │
│   │   ├── property/
│   │   │   ├── controller/PropertyController.java, BuildingController.java,
UnitController.java
│   │   │   ├── dto/...
│   │   │   ├── entity/Property.java, Building.java, Unit.java
│   │   │   ├── mapper/PropertyMapper.java
│   │   │   ├── repository/PropertyRepository.java, ...
│   │   │   └── service/PropertyService.java, UnitService.java
│   │   │
│   │   ├── lease/
│   │   │   ├── controller/LeaseController.java
│   │   │   ├── dto/CreateLeaseRequest.java, LeaseDto.java
│   │   │   ├── entity/Lease.java
│   │   │   ├── mapper/LeaseMapper.java
│   │   │   ├── repository/LeaseRepository.java
│   │   │   └── service/LeaseService.java
│   │   │
│   │   ├── billing/
│   │   │   ├── controller/InvoiceController.java, PaymentController.java
│   │   │   ├── dto/...
│   │   │   ├── entity/Invoice.java, Payment.java
│   │   │   ├── mapper/InvoiceMapper.java
│   │   │   ├── repository/InvoiceRepository.java, PaymentRepository.java
│   │   │   ├── scheduler/InvoiceGenerationJob.java
│   │   │   └── service/BillingService.java, PaymentService.java
│   │   │
│   │   ├── maintenance/
│   │   │   ├── controller/MaintenanceRequestController.java
│   │   │   ├── dto/...
│   │   │   ├── entity/MaintenanceRequest.java, MaintenanceAssignment.java
│   │   │   ├── mapper/MaintenanceMapper.java
│   │   │   ├── repository/MaintenanceRepository.java
│   │   │   └── service/MaintenanceService.java
│   │   │
│   │   ├── notification/
│   │   │   ├── controller/NotificationController.java
│   │   │   ├── dto/NotificationDto.java
│   │   │   ├── entity/Notification.java
│   │   │   ├── repository/NotificationRepository.java
│   │   │   ├── service/NotificationService.java
│   │   │   ├── consumer/LeaseEventConsumer.java
│   │   │   ├── consumer/PaymentEventConsumer.java
│   │   │   └── consumer/MaintenanceEventConsumer.java
│   │   │
│   │   └── reporting/
│   │       ├── controller/ReportController.java
│   │       ├── dto/OccupancyReportDto.java, RevenueReportDto.java
│   │       └── service/ReportService.java
│   │
│   ├── events/
│   │   ├── LeaseCreatedEvent.java
│   │   ├── PaymentCompletedEvent.java
│   │   ├── MaintenanceRequestedEvent.java
```

```
│   │   ├── MaintenanceStatusChangedEvent.java
│   │   └── LeaseExpiringEvent.java
│   │
│   ├── common/
│   │   ├── entity/BaseEntity.java       # id, createdAt, updatedAt, createdBy,
updatedBy
│   │   ├── dto/PagedResponse.java, ApiError.java
│   │   ├── exception/ResourceNotFoundException.java,
│   │   │             BusinessException.java, AccessDeniedException.java
│   │   └── handler/GlobalExceptionHandler.java
│   │
│   └── audit/
│       ├── entity/AuditLog.java
│       ├── repository/AuditLogRepository.java
│       └── service/AuditService.java
│
└── src/main/resources/
    ├── application.yml
    ├── application-dev.yml
    ├── application-prod.yml
    └── db/changelog/
        ├── db.changelog-master.xml
        └── v1.0.0/
            ├── 20240101-001-create-users.xml
            └── ...
```

11. Sample Implementation

11.1 Base Entity (JPA)

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @CreatedDate
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(name = "created_by", updatable = false)
    private Long createdBy;

    @LastModifiedBy
    @Column(name = "updated_by")
    private Long updatedBy;
}
```

11.2 Lease Entity

```
@Entity
@Table(name = "leases")
public class Lease extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "unit_id", nullable = false)
    private Unit unit;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tenant_id", nullable = false)
    private User tenant;

    @Column(name = "start_date", nullable = false)
    private LocalDate startDate;

    @Column(name = "end_date", nullable = false)
    private LocalDate endDate;

    @Column(name = "monthly_rent", nullable = false, precision = 12, scale = 2)
    private BigDecimal monthlyRent;

    @Column(name = "security_deposit", nullable = false, precision = 12, scale =
2)
    private BigDecimal securityDeposit = BigDecimal.ZERO;

    @Column(name = "payment_due_day", nullable = false)
    private int paymentDueDay = 1;

    @Column(name = "grace_period_days", nullable = false)
    private int gracePeriodDays = 5;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private LeaseStatus status = LeaseStatus.DRAFT;

    @Column(name = "terminated_at")
    private LocalDate terminatedAt;

    @OneToMany(mappedBy = "lease", cascade = CascadeType.ALL)
    private List<Invoice> invoices = new ArrayList<>();
}
```

11.3 Lease Repository

```
@Repository
public interface LeaseRepository extends JpaRepository<Lease, Long> {

    @Query("SELECT l FROM Lease l WHERE l.status = 'ACTIVE'
            AND l.endDate BETWEEN :start AND :end")
    List<Lease> findExpiringLeases(
        @Param("start") LocalDate start,
        @Param("end")   LocalDate end
    );

    @Query("SELECT l FROM Lease l WHERE l.tenant.id = :tenantId" +
           " AND l.status = ACTIVE")
    Optional<Lease> findActiveLease(@Param("tenantId") Long tenantId);
```

```
    boolean existsByUnitIdAndStatus(Long unitId, LeaseStatus status);

    Page<Lease> findByUnit_Building_Property_ManagerId(
        Long managerId, Pageable pageable
    );
}
```

11.4 Lease Service

```
@Service
@RequiredArgsConstructor
public class LeaseService {

    private final LeaseRepository   leaseRepository;
    private final UnitRepository     unitRepository;
    private final UserRepository     userRepository;
    private final LeaseMapper        leaseMapper;
    private final JmsTemplate        jmsTemplate;
    private final ObjectMapper       objectMapper;
    private final AuditService       auditService;

    @Transactional
    public LeaseDto createLease(CreateLeaseRequest req, Long currentUserId) {
        Unit unit = unitRepository.findById(req.getUnitId())
            .orElseThrow(() -> new ResourceNotFoundException("Unit",
req.getUnitId()));

        if (unit.getStatus() != UnitStatus.VACANT)
            throw new BusinessException("Unit is not available for lease");

        if (leaseRepository.existsByUnitIdAndStatus(unit.getId(),
LeaseStatus.ACTIVE))
            throw new BusinessException("Unit already has an active lease");

        User tenant = userRepository.findById(req.getTenantId())
            .orElseThrow(() -> new ResourceNotFoundException("User",
req.getTenantId()));

        Lease lease = leaseMapper.toEntity(req);
        lease.setUnit(unit);
        lease.setTenant(tenant);
        lease.setStatus(LeaseStatus.DRAFT);

        Lease saved = leaseRepository.save(lease);
        auditService.log("LEASE", saved.getId(), "CREATE", null, saved,
currentUserId);
        return leaseMapper.toDto(saved);
    }

    @Transactional
    public LeaseDto activateLease(Long leaseId, Long currentUserId) {
        Lease lease = leaseRepository.findById(leaseId)
            .orElseThrow(() -> new ResourceNotFoundException("Lease", leaseId));

        if (lease.getStatus() != LeaseStatus.DRAFT)
            throw new BusinessException("Only DRAFT leases can be activated");
```

```
        lease.setStatus(LeaseStatus.ACTIVE);
        lease.getUnit().setStatus(UnitStatus.OCCUPIED);
        leaseRepository.save(lease);

        LeaseCreatedEvent event = LeaseCreatedEvent.from(lease);
        jmsTemplate.convertAndSend("pms.lease.created",
            objectMapper.writeValueAsString(event));

        auditService.log("LEASE", leaseId, "STATUS_CHANGE",
            LeaseStatus.DRAFT, LeaseStatus.ACTIVE, currentUserId);
        return leaseMapper.toDto(lease);
    }
}
```

11.5 Lease Controller

```
@RestController
@RequestMapping("/api/leases")
@RequiredArgsConstructor
@Tag(name = "Leases")
public class LeaseController {

    private final LeaseService leaseService;

    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN', 'PROPERTY_MANAGER')")
    public ResponseEntity<LeaseDto> createLease(
            @Valid @RequestBody CreateLeaseRequest req,
            @AuthenticationPrincipal UserPrincipal principal) {
        LeaseDto dto = leaseService.createLease(req, principal.getId());
        URI location = URI.create("/api/leases/" + dto.getId());
        return ResponseEntity.created(location).body(dto);
    }

    @PatchMapping("/{id}/activate")
    @PreAuthorize("hasAnyRole('ADMIN', 'PROPERTY_MANAGER')")
    public ResponseEntity<LeaseDto> activateLease(
            @PathVariable Long id,
            @AuthenticationPrincipal UserPrincipal principal) {
        return ResponseEntity.ok(leaseService.activateLease(id,
principal.getId()));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('ADMIN','PROPERTY_MANAGER') or
                  (hasRole('TENANT') and @leaseService.isTenantOwner(#id,
principal.id))")
    public ResponseEntity<LeaseDto> getLease(@PathVariable Long id) {
        return ResponseEntity.ok(leaseService.getLeaseById(id));
    }

    @PatchMapping("/{id}/terminate")
    @PreAuthorize("hasAnyRole('ADMIN', 'PROPERTY_MANAGER')")
    public ResponseEntity<LeaseDto> terminateLease(
            @PathVariable Long id,
            @Valid @RequestBody TerminateLeaseRequest req,
```

```
            @AuthenticationPrincipal UserPrincipal principal) {
        return ResponseEntity.ok(leaseService.terminateLease(id, req,
principal.getId()));
    }
}
```

11.6 Event Publisher & Consumer

```
// ── Event POJO ──────────────────────────────────────────────
@Data @Builder
public class LeaseCreatedEvent {
    private String eventId;
    private String eventType = "LEASE_CREATED";
    private Instant timestamp;
    private Payload payload;

    @Data @Builder
    public static class Payload {
        private Long leaseId, unitId, tenantId, managerId;
        private LocalDate startDate, endDate;
        private BigDecimal monthlyRent;
    }

    public static LeaseCreatedEvent from(Lease l) {
        return LeaseCreatedEvent.builder()
            .eventId(UUID.randomUUID().toString())
            .timestamp(Instant.now())
            .payload(Payload.builder()
                .leaseId(l.getId()).unitId(l.getUnit().getId())
                .tenantId(l.getTenant().getId())
                .startDate(l.getStartDate()).endDate(l.getEndDate())
                .monthlyRent(l.getMonthlyRent()).build())
            .build();
    }
}

// ── Consumer ────────────────────────────────────────────────
@Component @RequiredArgsConstructor @Slf4j
public class LeaseEventConsumer {

    private final NotificationService notificationService;
    private final BillingService      billingService;
    private final ObjectMapper        objectMapper;

    @JmsListener(destination = "pms.lease.created",
                 containerFactory = "topicListenerFactory")
    public void onLeaseCreated(String raw) {
        try {
            LeaseCreatedEvent event =
                objectMapper.readValue(raw, LeaseCreatedEvent.class);
            log.info("Processing LeaseCreated eventId={}", event.getEventId());
            billingService.generateFirstInvoice(event.getPayload().getLeaseId());

notificationService.sendLeaseWelcome(event.getPayload().getTenantId());
        } catch (Exception ex) {
            log.error("Failed to process LeaseCreatedEvent", ex);
            throw new RuntimeException(ex); // triggers ActiveMQ redelivery
        }
```

```
        }
    }
```

12. Future Enhancements

12.1 Multi-Tenancy

- Introduce a tenant_id (company ID) discriminator column on all entities
- Row-Level Security (RLS) via Hibernate Filter: @FilterDef + @Filter applied in session
- Separate schema per company is an alternative (higher isolation, higher ops cost)
- API key + JWT claim to carry company_id; all service queries automatically scoped

12.2 Payment Gateway Integration

Gateway	Integration Approach
Stripe	Stripe Checkout Sessions for online rent payment; webhooks for payment events → PaymentService
PayPal	PayPal Orders API v2; IPN/webhook handler maps to Invoice payment recording
Fawry (Egypt)	Fawry Pay API for local Egyptian market cash/card payments; reference code issued to tenant

12.3 Microservices Migration Path

- Extract Billing Service first (highest independent scalability need)
- Extract Notification Service (async by nature, natural boundary)
- Introduce API Gateway (Spring Cloud Gateway) for routing and cross-cutting concerns
- Add distributed tracing (Zipkin/Jaeger) and centralized logging (ELK stack)
- Use Saga pattern (Orchestration via Camunda/Temporal) for distributed transactions

12.4 Mobile App Support

- Current REST API is already mobile-ready; add GraphQL layer for flexible querying
- Push notifications: integrate FCM (Firebase Cloud Messaging) into NotificationService
- Offline-first: mobile clients sync lease/payment data with conflict resolution strategy
- Biometric authentication: FIDO2 / WebAuthn for passwordless login

12.5 Additional V2 Features

- Document Management: lease PDF generation (JasperReports), e-signature integration
- Calendar Integration: maintenance scheduling with Google Calendar API
- AI-powered rent pricing suggestions based on market data
- Tenant portal as a standalone React/Vue SPA with dedicated BFF service
- Advanced analytics dashboard with Grafana + Prometheus metrics

— End of Technical Documentation —

Property Management System v1.0 | Prepared by Senior Architecture Team